

洛谷数列分块入门 1 - 9 总结

这组题的核心不是某一种固定数据结构，而是同一个思想：

把长度为 n 的序列分成若干块。对区间操作时，两端不完整的块暴力处理，中间完整块用预处理信息或懒标记快速处理。

如果块长设为 B ，块数大约是 n / B 。一次普通区间操作通常会花：

$O(B)$ 处理左右散块
 $O(n / B)$ 处理中间整块

所以一般取：

$B \approx \sqrt{n}$

这套题的递进关系很清楚：

- 入门 1、4、7：维护懒标记。
- 入门 2、3：块内排序 + 二分。
- 入门 5：无法快速整块开方，于是利用“开方次数很少”的均摊性质。
- 入门 6：块状数组，处理插入和询问。
- 入门 8：区间覆盖 + 查询等于某值的个数，靠覆盖标记和均摊。
- 入门 9：静态区间众数，靠块间预处理和散块补充。

通用模板思想

常见数组：

```
int belong[i];    // i 属于哪一块
int L[id], R[id]; // 第 id 块左右端点
```

初始化：

```
int B = sqrt(n);
int tot = (n + B - 1) / B;

for (int id = 1; id <= tot; id++) {
    L[id] = (id - 1) * B + 1;
    R[id] = min(id * B, n);
    for (int i = L[id]; i <= R[id]; i++) {
        belong[i] = id;
    }
}
```

区间操作 $[l, r]$ 的基本结构：

```
int b1 = belong[l], br = belong[r];

if (b1 == br) {
    // 同一块内，直接暴力
} else {
    // 左散块 [l, R[b1]]
    // 右散块 [L[br], r]
    // 中间整块 b1 + 1 到 br - 1
}
```

分块题的关键问题通常有三个：

- 1. 散块怎么暴力处理？
- 2. 整块怎么快速处理？
- 3. 散块暴力后，块的信息怎么重构？

总览对比

题号	主要操作	维护信息	核心技巧	复杂度印象
入门 1	区间加，单点查询	块加标记 <code>tag</code>	整块懒加，散块暴力	$O(\sqrt{n})$
入门 2	区间加，查询 $< x$ 个数	块加标记 + 块内有序数组	整块二分，散块暴力后重排	$O(\sqrt{n} \log n)$
入门 3	区间加，查询 $< x$ 前驱	块加标记 + 块内有序数组	二分找前驱	$O(\sqrt{n} \log n)$
入门 4	区间加，区间求和	块加标记 + 块和 <code>sum</code>	整块直接加贡献	$O(\sqrt{n})$
入门 5	区间开方，区间求和	块和 + 是否稳定标记	每个数很快变成 0/1，均摊暴力	近似 $O(n \sqrt{n})$
入门 6	单点插入，单点询问	每块动态数组	块状数组 + 定期重构	均摊 $O(\sqrt{n})$
入门 7	区间乘，区间加，单点查询	乘法标记 + 加法标记	维护仿射变换 $x \rightarrow mul * x + add$	$O(\sqrt{n})$
入门 8	查询等于 <code>c</code> 的个数，并覆盖为 <code>c</code>	块覆盖标记	同色块 $O(1)$ ，散块下放后暴力	均摊 $O(\sqrt{n})$
入门 9	静态区间最小众数	值离散化、位置表、块间众数	预处理整块答案，散块补候选	查询约 $O(\sqrt{n} \log n)$

入门 1：区间加法，单点查询

相似点

这是最基础的“整块打懒标记，散块暴力”模型。后面的 4、7 都是在它的基础上加强懒标记。

思路

维护：

```
long long a[i];           // 原数组，散块修改时直接改
long long tag[id];        // 第 id 块整体加了多少
```

区间加 `[l, r] += c`：

- 如果在同一块，直接 `a[i] += c`。
- 否则左右散块暴力加。
- 中间整块只改 `tag[id] += c`。

单点查询 `a[x]`：

```
a[x] + tag[belong[x]]
```

易错点

- 散块修改的是 `a[i]`，整块修改的是 `tag`。
- 查询时一定要加上块标记。

入门 2：区间加法，查询区间内小于 x 的个数

相似点

和入门 1 一样支持区间加，但查询不是单点，而是区间统计。

因此仅有 `tag` 不够，还要让每个块支持快速计数。

思路

维护：

```
long long a[i];           // 原数组
long long tag[id];        // 块加标记
vector<long long> b[id];  // 每块排序后的值
```

查询 `< x`：

- 散块：暴力枚举 `a[i] + tag[belong[i]] < x`。
- 整块：块内所有数都加了同一个 `tag[id]`，所以要统计：

`b[id]` 中 $< x - \text{tag}[id]$ 的数量

用 `lower_bound`。

区间加：

- 整块： `tag[id] += c`。
- 散块：先修改 `a[i] += c`，然后重构这一块的有序数组。

重构

```
void rebuild(int id) {
    b[id].clear();
    for (int i = L[id]; i <= R[id]; i++) b[id].push_back(a[i]);
    sort(b[id].begin(), b[id].end());
}
```

易错点

- 块内排序数组存的是不含 `tag` 的原值。
- 二分时查 `x - tag[id]`，不是直接查 `x`。
- 散块修改后必须 `rebuild`。

入门 3：区间加法，查询区间内小于 x 的前驱

相似点

和入门 2 几乎一样。区别只是查询目标从“个数”变成“最大的小于 x 的数”。

思路

维护方式仍然是：

`a[i]`, `tag[id]`, sorted block

查询 `[l, r]` 中 $< x$ 的最大值：

- 散块暴力枚举实际值 `a[i] + tag[belong[i]]`。
- 整块用二分：

```
auto it = lower_bound(b[id].begin(), b[id].end(), x - tag[id]);
```

如果 `it != begin`，则候选为：

```
*(--it) + tag[id]
```

易错点

- 要找的是严格小于 x ，所以用 `lower_bound`，不能用 `upper_bound`。
- 不存在答案时输出 `-1`。
- 仍然要注意散块修改后的重构。

入门 4：区间加法，区间求和

相似点

和入门 1 一样是区间加。区别是查询从单点变成区间和，所以每块要额外维护块和。

思路

维护：

```
long long a[i];
long long tag[id];
long long sum[id]; // 块内元素实际总和
```

区间加 `[l, r] += c`：

- 散块： `a[i] += c`，同时 `sum[belong[i]] += c`。
- 整块：

```
tag[id] += c;
sum[id] += c * blockSize;
```

区间求和：

- 散块累加 `a[i] + tag[belong[i]]`。
- 整块直接加 `sum[id]`。

如果题面要求输出 `sum % (c + 1)`，最后再取模即可。

易错点

- `sum[id]` 应该表示“当前真实块和”，整块加时也要更新。
 - 如果 `sum` 不含 `tag`，查询公式会变复杂；初学建议让 `sum` 始终维护真实值。
-

入门 5：区间开方，区间求和

不同点

这题没有好用的整块懒标记。

因为：

```
sqrt(a + tag) 不等于 sqrt(a) + 某个统一标记
```

所以不能像区间加那样整块 $O(1)$ 更新。

思路

利用性质：

一个非负整数连续开方取整几次后，会很快变成 0 或 1。

维护：

```
long long a[i];
long long sum[id];
bool stable[id]; // 这一块是否所有数都已经是 0 或 1
```

区间开方：

- 散块：暴力 `a[i] = sqrt(a[i])`，更新块和。
- 整块：
 - 如果 `stable[id] == true`，跳过。
 - 否则暴力整块开方，然后重算块和和稳定标记。

区间求和：

- 散块暴力。
- 整块加 `sum[id]`。

为什么能过

虽然整块开方看起来可能是 $O(n \sqrt{n})$ ，但每个元素最多被有效开方很少次。

一旦变成 0/1，之后都不会再变，所在块也会逐渐被跳过。

易错点

- `sqrt` 用 `long long` 时建议写：

```
long long v = sqrt(a[i]);
while ((v + 1) * (v + 1) <= a[i]) v++;
while (v * v > a[i]) v--;
```

- 每次暴力改块后，要重算 `sum` 和 `stable`。

入门 6：单点插入，单点询问

不同点

前面几题数组下标基本固定，这题会插入元素，序列长度变化。
这时普通数组不好维护，所以用“块状数组”。

思路

每一块用 `vector<int>` 存当前块内元素。

查询第 `k` 个元素：

从前往后减去每块大小，找到所在块，再在块内访问。

插入到第 `k` 个位置：

同样先找到所在块，然后：

```
vec[id].insert(vec[id].begin() + offset, x);
```

重构

如果一直往同一块插，某个块可能变得很大。
所以需要定期重构：

- 每隔大约 `sqrt(n)` 次插入，重新把所有元素摊平成数组。
- 再按块长重新分块。

或者：

- 当某块大小超过 `2 * B` 时，把这一块拆成两块。

易错点

- 插入位置是“当前序列”的位置，不是原数组位置。
- 找第 `k` 个元素时，注意 `k` 是 1-indexed。
- 重构时要更新当前总长度。

入门 7：区间乘法，区间加法，单点询问

相似点

它是入门 1 的加强版。
入门 1 的块标记只有：

```
x -> x + add
```

入门 7 的块标记变成：

```
x -> mul * x + add
```

思路

维护：

```
long long a[i];
long long mulTag[id];
long long addTag[id];
```

初始：

```
mulTag[id] = 1;
addTag[id] = 0;
```

整块乘 c ：

```
mulTag[id] *= c;
addTag[id] *= c;
```

整块加 c ：

```
addTag[id] += c;
```

单点真实值：

```
a[i] * mulTag[belong[i]] + addTag[belong[i]]
```

散块修改前，最好先下放标记：

```
void pushDown(int id) {
    for (int i = L[id]; i <= R[id]; i++) {
        a[i] = a[i] * mulTag[id] + addTag[id];
    }
    mulTag[id] = 1;
    addTag[id] = 0;
}
```

如果题目要求模 10007，所有操作都在模意义下完成。

易错点

- 乘法会影响之前的加法标记：

$$(x * mul + add) * c = x * (mul * c) + (add * c)$$

- 标记下放后一定要清空。
- 先乘后加、先加后乘不是一回事。

入门 8：区间查询等于 c 的个数，并覆盖为 c

不同点

这个题每次操作既是查询也是修改：

问 $[l, r]$ 里等于 c 的个数，然后把 $[l, r]$ 全部改成 c

它不像入门 2 那样适合块内排序，因为修改是覆盖，值域也可能很大。
更自然的是维护“这一块是否全是同一个值”。

思路

维护：

```
long long a[i];
long long tag[id];
bool same[id]; // same[id] 表示整块是否都等于 tag[id]
```

整块操作：

- 如果 `same[id] == true`：
 - 若 `tag[id] == c`，这一整块全部贡献答案。
 - 否则贡献 0。
 - 然后整块覆盖成 c ，即 `tag[id] = c`。
- 如果 `same[id] == false`：
 - 暴力统计并覆盖整块。
 - 覆盖后 `same[id] = true, tag[id] = c`。

散块操作：

先下放：

```
if (same[id]) {
    for (int i = L[id]; i <= R[id]; i++) a[i] = tag[id];
}
same[id] = false;
```

然后暴力统计 $[l, r]$ 中等于 c 的位置，并赋值为 c 。

最后检查这个块是否又变成全相同。

为什么复杂度可以接受

一个操作真正暴力扫很多块，通常是因为这些块不是同色块。

但一次区间覆盖会把大量块重新变成同色块。

散块只会破坏左右两个块，因此整体可以用均摊分析通过。

易错点

- 散块操作前必须下放同色标记。
- 散块修改后要重新判断这一块是否变成同色。
- 操作顺序是：先统计等于 c 的个数，再覆盖为 c 。不过因为覆盖值也是 c ，边统计边覆盖即可。

入门 9：静态区间最小众数

不同点

这是 9 题中最不一样的一题：

- 没有修改。
- 查询区间众数。
- 众数不可直接由左右区间答案合并得到。
- 要求相同次数时取最小值。

思路

由于没有修改，可以大量预处理。

第一步：离散化。

```
原值 -> 编号  
编号 -> 原值
```

第二步：记录每个值出现的位置。

```
pos[valueId] = 所有出现下标，递增
```

这样可以用二分求某个值在 $[l, r]$ 中出现了几次：

```
cnt(x, l, r) =  
upper_bound(pos[x].begin(), pos[x].end(), r)  
- lower_bound(pos[x].begin(), pos[x].end(), l)
```

第三步：预处理整块区间的最小众数。

令：

```
mode[i][j] = 从第 i 块到第 j 块这一整段的最小众数
```

枚举左块 i ，向右扩展块 j ，用计数数组维护当前众数。

查询 $[l, r]$ ：

- 如果在同一块，暴力枚举候选。
- 否则：

- 中间完整块 `[belong[l]+1, belong[r]-1]` 的答案先取 `mode`。
- 左右散块里的每个元素也作为候选。
- 对每个候选，用 `pos` 二分统计它在 `[l, r]` 的出现次数。
- 按“次数更大优先，次数相同值更小优先”更新答案。

为什么只检查散块候选就够

真正的众数如果不在中间整块答案里，那它想打败中间答案，额外贡献只能来自左右散块。因此它必然至少在散块出现过。

所以候选集合是：

中间整块众数 + 左散块元素 + 右散块元素

易错点

- 众数相同时要输出原值更小的那个，不是离散编号更小，除非离散编号按原值排序。
- 查询时散块候选可能重复，重复检查不影响正确性，但可以用临时标记去重。
- 强制在线版本常见写法会让输入的 `l, r` 与上一次答案有关，要先还原真实区间。

横向比较：这些题到底像在哪里

都在处理“散块 + 整块”

每道题几乎都可以先写出这个框架：

如果 `l, r` 在同一块：暴力
否则：
暴力左散块
暴力右散块
快速处理中间整块

真正不同的是“整块快速处理”的方式。

都需要明确块信息是否包含懒标记

例如：

- 入门 2、3 的有序数组一般不含 `tag`，查询时用 `x - tag`。
- 入门 4 的 `sum` 建议维护真实块和，整块加时直接更新。
- 入门 7 的 `a[i]` 可以存未下放值，真实值由 `mulTag/addTag` 算出。

写代码前一定要先定清楚：

`a[i]` 是真实值，还是未下放值？
块信息是否已经包含 `tag`？

散块修改后常常要重构

典型题：

- 入门 2、3：散块改值后重排该块。
- 入门 5：散块开方后重算块和和稳定标记。
- 入门 8：散块覆盖后重新判断是否同色。

很多错误都来自“散块改了，但块信息没更新”。

横向比较：这些题不同在哪里

1、4、7：懒标记型

这类题最顺手，整块操作能写成统一变换。

- 入门 1： $x \rightarrow x + c$
- 入门 4： $x \rightarrow x + c$ ，额外维护和
- 入门 7： $x \rightarrow x * c$ 或 $x \rightarrow x + c$

适合用线段树，也适合用分块。

2、3：块内有序型

这类题要求整块内部可以二分。

- 入门 2 查询个数。
- 入门 3 查询前驱。

本质是：

块内排序 + 整块统一加法不改变相对顺序

5、8：均摊暴力型

这类题不容易维护整块信息，但操作有“越做越简单”的性质。

- 入门 5：数开方后很快变成 0/1。
- 入门 8：覆盖后整块容易变成同一个值。

它们不是靠每次严格 $O(\sqrt{n})$ ，而是靠总暴力次数可控。

6：块状数组型

它不再只是维护区间信息，而是维护一个会动态变长的序列。

关键是：

块内 `vector` 暴力插入 + 定期重构

9：静态预处理型

它没有修改，所以可以预处理块区间答案。
查询时用整块答案加散块候选补全。

代码实现建议

块长怎么选

一般：

```
int B = sqrt(n);
```

但实际常数不同：

- 入门 2、3 有二分和重构，块长可以略大。
- 入门 9 预处理 `mode`，块数不能太多，常取 `B ≈ sqrt(n)` 或稍大。
- 入门 6 动态插入，重构阈值可以取 `sqrt(当前长度)` 或固定块长。

比赛中如果卡常，可以试：

```
int B = 700; // n around 1e5
```

或者对拍不同块长。

下放函数很重要

遇到懒标记题，建议写：

```
void pushDown(int id)
```

尤其是入门 7、8。

散块暴力前先下放，代码更不容易乱。

重构函数也要单独写

例如：

```
void rebuild(int id)
```

用于：

- 重新排序块内数组。
- 重算块和。
- 重算同色标记。

把重构封装出来，散块修改后调用，能少很多边界错误。

推荐学习顺序

如果是第一次学分块，建议：

1. 入门 1：理解散块和整块。
2. 入门 4：在入门 1 基础上维护块和。
3. 入门 2：学习块内排序和二分。
4. 入门 3：把二分查询从计数改成前驱。
5. 入门 7：学习多个懒标记如何组合。
6. 入门 5：理解均摊暴力。
7. 入门 8：理解覆盖标记和均摊。
8. 入门 6：学习动态块状数组。
9. 入门 9：学习静态块间预处理。

常见错误清单

- `belong[l] == belong[r]` 时忘记特判。
- 整块下标写成 `bl` 到 `br`，把左右散块重复处理。
- 散块修改后忘记 `rebuild`。
- 查询时忘记加 `tag`。
- 二分时忘记减 `tag`。
- `sum[id]` 到底是否包含 `tag` 没想清楚。
- 入门 7 中乘法标记和加法标记组合顺序写反。
- 入门 8 中散块修改前忘记下放覆盖标记。
- 入门 9 中众数次数相同要取最小值。

参考资料

- hzwer：《数列分块入门 1 - 9》：<https://hzwer.com/8053.html>
- OI Wiki：分块思想：<https://oi-wiki.org/ds/decompose/>
- LibreOJ 题面集合：#6277 - #6285 数列分块入门